Machine Learning Projects

Data Import → EDA → Preprocessing → Model-building → Evaluation → Saving

Flight Fare Prediction — Regression

1.1 Objective

Predict flight ticket prices (Price) from trip attributes such as airline, source, destination, route, departure/arrival time, duration, number of stops, and journey date. This is a supervised regression problem on tabular data.

1.2 Data Import

The dataset is loaded from a local Excel file:

data = pd.read_excel("Flight_Fare.xlsx")

Shape: 10,683 rows × 11 columns. Columns: Airline, Date_of_Journey, Source, Destination, Route, Dep_Time, Arrival_Time, Duration, Total_Stops, Additional_Info, Price (target). All feature columns are text/object type except Price.

1.3 Exploratory Data Analysis (EDA)

- Checked dtypes, describe(), nunique() per column, and unique category values for every categorical field.
- Visualized distributions with count plots (Airline, Total_Stops), histograms (Journey_Month, Date_of_Journey, Price), a violin plot and KDE plot for Price, scatter plots (Price vs Airline, Date_of_Journey vs Price colored by Airline), a full pairplot, and a correlation heatmap on numeric columns.
- Box plots and histograms were generated for every numeric column to inspect spread and skew.
- Found Price is right-skewed (skew ≈ 1.86) before treatment.

1.4 Data Cleaning

- **Duplicates:** 220 duplicate rows detected and removed with drop_duplicates() (10,683 → 10,463 rows).
- **Missing values:** 1 missing Route and 1 missing Total_Stops — dropped via dropna() (final 10,462 rows). At <0.02% of the data, dropping was reasonable rather than imputing.
- **Outliers in Price:** IQR method flagged ≈0.9% of rows as outliers. These were replaced with the column mean, which brought skew down from 1.86 to 0.45. A log transform (np.log) was then also applied to Price for the modeling target.

1.5 Feature Engineering

- Date_of_Journey parsed to datetime, then split into Journey_Day, Journey_Month, Journey_Year.
- Dep_Time parsed to Dep_Hour (int) and bucketed into Dep_Period (Early Morning / Morning / Afternoon / Evening) using a custom function.
- Duration string (e.g. "2h 50m") converted to Duration_Minutes via a custom parser.
- Dep_Time and Arrival_Time converted to Dep_Time_Minutes / Arrival_Time_Minutes.
- Arrival_Time (which sometimes embeds a date, e.g. "01:10 22 Mar") split into Arrival_Date, Arrival_Month, and a cleaned Arrival_Time.

1.6 Encoding & Scaling

- LabelEncoder applied to all remaining categorical columns: Arrival_Date, Airline, Source, Destination, Route, Dep_Time, Duration, Total_Stops, Additional_Info, Dep_Period, Arrival_Month, Arrival_Time.
- MinMaxScaler applied to Duration_Minutes and Dep_Time_Minutes.

1.7 Train/Test Split

```
x = data.drop(columns=['Price','Dep_Time','Arrival_Time','Duration',
                       'Additional_Info','Date_of_Journey'])
y = data['Price']
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

Result: 8,369 training rows / 2,093 test rows, 15 feature columns.

1.8 Model Building, Training & Tuning

Six regression algorithms were trained and compared on the same split, several with GridSearchCV / RandomizedSearchCV (5-fold CV):

Model	Tuning	R^2	MAE	RMSE
Linear Regression	None	0.567	0.268	0.328
K-Nearest Neighbors (k=3)	Manual	0.754	0.175	0.247
Decision Tree	Default	0.767	0.141	0.240
Decision Tree	Manual params	0.721	0.208	0.263
Random Forest	Default	0.853	0.124	0.191
Random Forest	Manual params	0.778	0.188	0.235

Gradient Boosting	Default	0.821	0.164	0.211
Gradient Boosting	GridSearchCV	0.890	0.121	0.166
Gradient Boosting	RandomizedSearchCV	0.888	0.117	0.167
XGBoost	Default	0.888	0.116	0.167
SVR	Default	0.151	0.376	0.459

Best model: Gradient Boosting Regressor tuned via GridSearchCV (learning_rate=0.1, max_depth=5, n_estimators=300), $R^2 \approx 0.89$ on the log-transformed Price target.

1.9 Save & Reload the Model

```
import pickle
pickle.dump(model, open('gradient_boosting.pkl', 'wb'))
```

Note: the pickle call in the notebook actually runs right after the last SVR cell, so double-check which fitted `model` object is in memory at that point before relying on the saved file — see Challenges below.

1.10 Pros

- Strong, systematic feature engineering: raw date/time strings are decomposed into numeric and cyclical-style features (day, month, hour, period-of-day) that tree models can split on effectively.
- Wide EDA coverage — correlation heatmap, pairplot, and per-column distribution/box plots make it easy to spot skew and relationships before modeling.
- Six different algorithm families compared like-for-like on an identical split, giving a fair leaderboard.
- Both GridSearchCV and RandomizedSearchCV used, and outcomes cross-checked against each other.
- Log-transforming the skewed target is the right call for linear/gradient-based losses.

1.11 Cons, Challenges & How to Handle Them

- **Bug — misspelled class name:** several cells call XGRegressor(...) instead of XGBRegressor(...) (cells using the tuned XGBoost params). This raises a NameError if run in a fresh kernel. Fix: use the imported XGBRegressor consistently.
- **Bug — invalid SVR parameters:** SVR(learning_rate=0.1, max_depth=5, n_estimators=300) passes RandomForest/GBM-style arguments into SVR, which doesn't accept them — this throws a TypeError. Fix: only pass SVR's own hyperparameters (C, kernel, gamma, epsilon).

- **Outlier treatment distorts the distribution:** replacing outliers with the column mean creates an artificial spike of identical values rather than preserving the shape of the tail. A more standard approach is winsorizing (cap at the IQR bounds) or relying on the log transform alone to tame skew.
- **High-cardinality label encoding:** Route (128 categories) and Arrival_Time (over a thousand raw values before bucketing) are label-encoded, which imposes a false ordinal relationship on nominal data. Tree-based models tolerate this reasonably well, but linear/SVR models are misled by it. Frequency or target encoding would be safer for those columns.
- **Partial scaling:** only two numeric columns are scaled; every other numeric feature (encoded categories, minute-of-day values) is left on its raw scale. This mainly hurts distance/gradient-based models like KNN and SVR — consistent with their poor scores above. Fix: scale all numeric features together, or use models that are scale-invariant (tree ensembles) when scaling every column isn't practical.
- **Best params not always carried through:** in a couple of places the manually re-typed hyperparameters after a search (e.g. max_depth) don't exactly match the printed best_params_. Fix: refit directly from grid.best_estimator_ / random.best_estimator_ instead of retyping values.
- **Ambiguous state at save time:** the final pickle.dump(model, ...) saves whatever `model` variable happens to be in memory — because SVR was fit last (and would itself have errored), it's easy to accidentally persist the wrong model. Fix: give each model a distinct variable name (e.g. gbm_best) and pickle that explicitly.
- **No reload/verification step:** the plan calls for reloading the saved model and predicting again to prove it works, but this cell isn't present in the notebook. Add: pickle.load(...) followed by a predict() call on a held-out sample.

Streamlit UI not built: listed as an optional step in the plan but not implemented in this notebook — a natural next iteration